

Serial Port Implementation Details

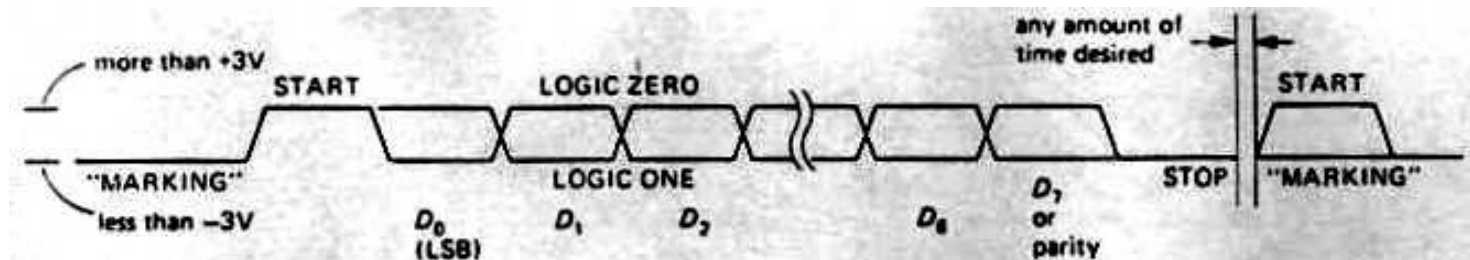
(1st Lab Work)

FEUP

Computer Networks

2026/2027

- » Each character is delimited by
 - Start bit
 - Stop bit (typically 1 or 2)
- » Each character consists of 8 bits (D0 – D7)
- » Parity
 - Even – even number of 1s
 - Odd – uneven number of 1s
 - Inhibited (D7 bit used for data) – option adopted in this link layer protocol
- » Transmission rate: 300 a 115200 bit/s



- Physical layer protocol between a computer or terminal (DTE) and modem (DCE)
 - » DTE (*Data Terminal Equipment*)
 - » DCE (*Data Circuit-Terminating Equipment*)

Connectors DB25 e DB9

Active signal

Control signal ($> + 3 \text{ V}$)

Data signal ($< - 3 \text{ V}$)

DTR (Data Terminal Ready) – Computer on

DSR (Data Set Ready) – Modem on

DCD (Data Carrier Detected) – Modem

detects phone line carrier

RI (Ring Indicator) – Modem detects ring

RTS (Request to Send) – Computer ready to

communicate

CTS (Clear To Send) – Modem ready to

communicate

TD (Transmit data) – Data transmission

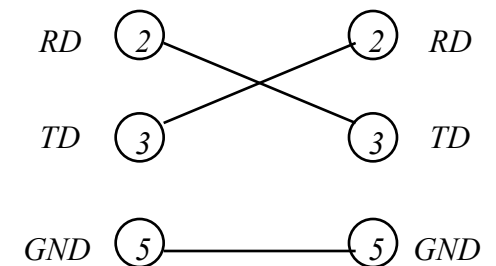
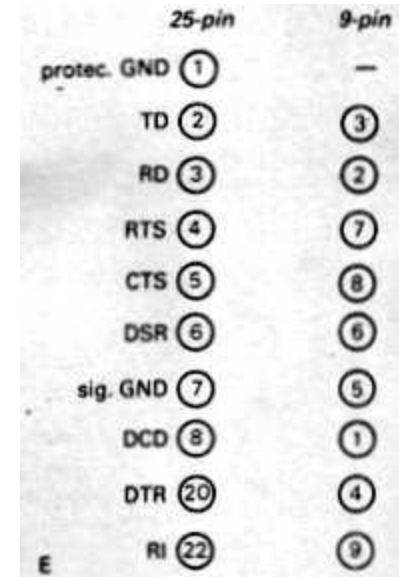
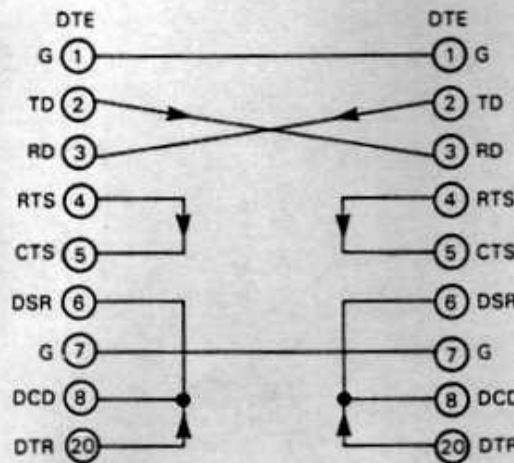
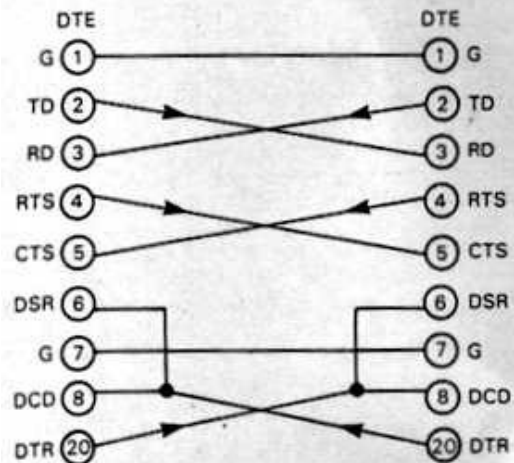
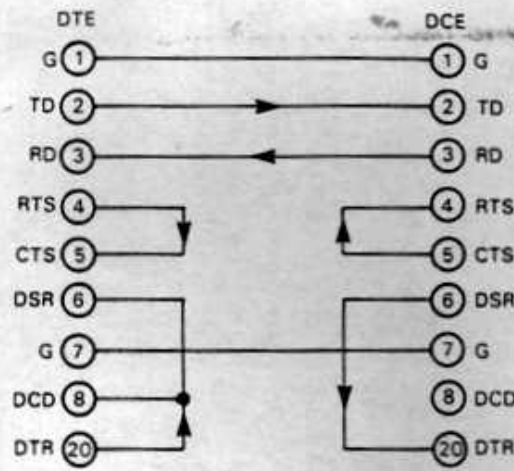
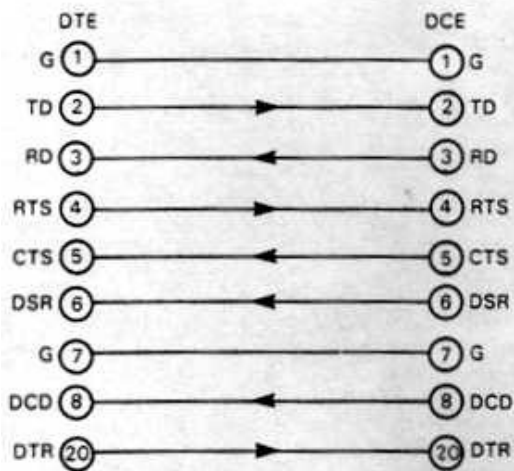
RD (Receive data) – Data reception

TABLE 10.4. RS-232 SIGNALS

Name	Pin number		Direction (DTE↔DCE)	Function (as seen by DTE)	
	25-pin	9-pin			
TD	2	3	→	transmitted data	} data pair
RD	3	2	←	received data	
RTS	4	7	→	request to send (= DTE ready)	} handshake pair
CTS	5	8	←	clear to send (= DCE ready)	
DTR	20	4	→	data terminal ready	} handshake pair
DSR	6	6	←	data set ready	
DCD	8	1	←	data carrier detect	} enable DTE input
RI	22	9	←	ring indicator	
FG	1	–		frame ground (= chassis)	
SG	7	5		signal ground	

Connections between equipment

4



Null Modem (9 pinos)

» Characteristics

- Software that manages a hardware controller
- Set of low level routines with privileged execution access
- Reside in memory (they are part of the kernel)
- Hardware interruption associated

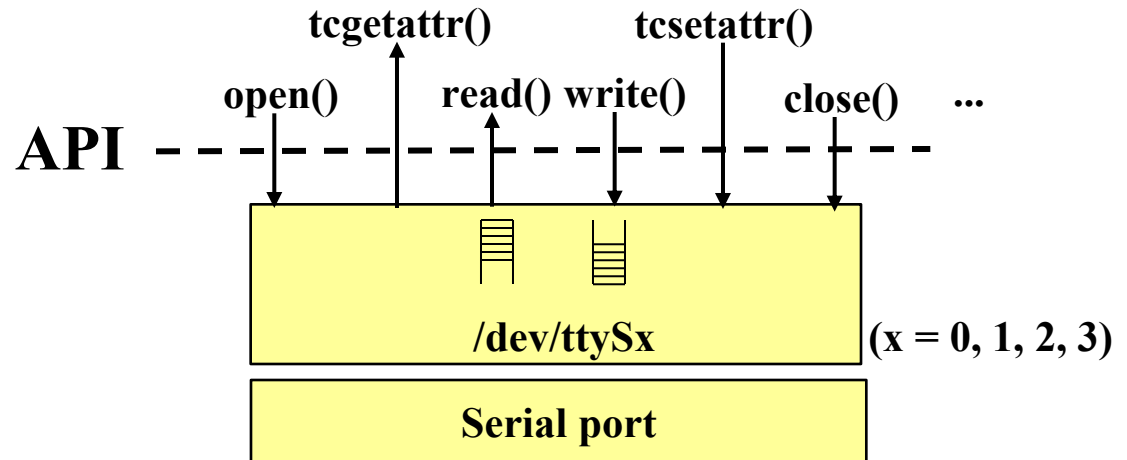
» Access method

- Mapped into Unix file system (/dev/hda1, /dev/ttyS0)
- Offered services similar to files (*open, close, read, write*)

» Driver types

- Character
 - Read and write from the controlled as multiple characters
 - Direct access (data is not stored in buffers)
- Block
 - Read/write as multiples of a block (block = 512 or 1024 octets)
 - Data sorted in buffers and random access
- Network
 - Read and write variable size packets
 - Sockets interface

API – Application Programming Interface



Some API functions

```
int open (DEVICE, O_RDWR);           /*returns a file descriptor*/
int read (int fileDescriptor, char * buffer, int numChars); /*returns the number of characters read*/
int write (int fileDescriptor, char * buffer, int numChars); /*returns the number of characters written*/
int close (int fileDescriptor);

int tcgetattr (int fileDescriptor, struct termios *termios_p);
int tcflush (int fileDescriptor, int Queueselector);      /*TCIFLUSH, TCOFLUSH ou TCIOFLUSH*/
int tcsetattr (int fileDescriptor, int modo, struct termios *termios_p);
```

termios data structure – allows to configure and store the serial port configuration parameters

```
struct termios {
    tcflag_t    c_iflag;    /*reception configuration flags*/
    tcflag_t    c_oflag;    /*transmission configuration flags*/
    tcflag_t    c_cflag;    /*control flags*/
    tcflag_t    c_lflag;    /*local configuration flags*/
    cc_t        c_line;     /*not used*/
    cc_t        c_cc[NCCS]  /*control characteres; NCCS = 19*/
};
```

Example:

```
#define BAUDRATE B38400
struct termios newtio;

/* CS8:      8n1 (8 bits, without parity bit, 1 stopbit)*/
/* CLOCAL:   local connection, without modem*/
/* CREAD:    enables the reception of characters*/
newtio.c_cflag = BAUDRATE | CS8 | CLOCAL | CREAD;

/* IGNPAR:   Ignores parity bits errors*/
/* ICRNL:    Converts CR into NL*/
newtio.c_iflag = IGNPAR | ICRNL;

newtio.c_oflag = 0;    /*Output not processed*/

/* ICANON:   enables the canonic reception */
newtio.c_lflag = ICANON;
```

- **Canonic**
 - » *read()* returns only full lines (ended by ASCII LF, EOF, EOL)
 - » Used for terminals
- **Non-canonic**
 - » *read ()* returns up to a maximum number of characters
 - » Enables the configuration of a maximum time between each character read
 - » Suitable for reading groups of characters
- **Asynchronous**
 - » *read()* returns immediately
 - » Uses a *signal handler*

Canonic Reception

```
main() {

    int fd,c, res;
    struct termios oldtio,newtio;
    char buf[255];

    fd = open(/dev/ttyS1,O_RDONLY|O_NOCTTY);
    tcgetattr(fd,&oldtio);

    bzero(&newtio, sizeof(newtio));
    newtio.c_cflag = B38400|CS8|CLOCAL|CREAD;
    newtio.c_iflag = IGNPAR|ICRNL;
    newtio.c_oflag = 0;
    newtio.c_lflag = ICANON;
    tcflush(fd, TCIFLUSH);
    tcsetattr(fd,TCSANOW,&newtio);

    res = read(fd,buf,255);

    tcsetattr(fd,TCSANOW,&oldtio);
    close(fd);
}
```

Non-canonic Reception

```
main() {

    int fd,c, res;
    struct termios oldtio,newtio;
    char buf[255];

    fd = open(argv[1], O_RDWR | O_NOCTTY );
    tcgetattr(fd,&oldtio);

    bzero(&newtio, sizeof(newtio));
    newtio.c_cflag = B38400 | CS8 | CLOCAL | CREAD;
    newtio.c_iflag = IGNPAR;
    newtio.c_oflag = 0;
    newtio.c_lflag = 0;
    newtio.c_cc[VTIME] = 0; /* timer between characters
    newtio.c_cc[VMIN] = 5; /* block until 5 characters
    are read */

    tcflush(fd, TCIFLUSH);
    tcsetattr(fd,TCSANOW,&newtio);

    res = read(fd,buf,255); /* at least 5 characters */

    tcsetattr(fd,TCSANOW,&oldtio);
    close(fd);
}
```

Asynchronous Reception

```
void signal_handler_IO (int status); /* signal
handler definition */
main() {
    /* variables declaration and serial port open */
    saio.sa_handler = signal_handler_IO;
    saio.sa_flags = 0;
    saio.sa_restorer = NULL; /* obsolete */
    sigaction(SIGIO,&saio,NULL);
    fcntl(fd, F_SETOWN, getpid());
    fcntl(fd, F_SETFL, FASYNC);
    /* serial port configuration through the termios
    structure */
    while (loop) {
        write(1, ".", 1);usleep(100000);
        /* after SIGIO signal, wait_flag = FALSE, data
        available to read */
        if (wait_flag==FALSE) {
            read(fd,buf,255); wait_flag = TRUE; /*
            waiting for new data to be read */
        }
    }
    /* configure the serial port with the initial
    values and close */
}
void signal_handler_IO (int status) { wait_flag =
FALSE; }
```

Multiple Reception

```
main(){
    int fd1, fd2; /*input sources 1 and 2*/
    fd_set readfs; /*file descriptor set */
    int maxfd, loop = 1; int loop=TRUE;
    /* open_input_source opens a device, sets the
    port correctly, and returns a file descriptor */
    fd1 = open_input_source("/dev/ttyS1"); /*
    COM2 */
    fd2 = open_input_source("/dev/ttyS2"); /*
    COM3 */
    maxfd = MAX (fd1, fd2)+1; /*max bit entry
    (fd) to test*/
    while (loop) { /* loop for input */
        FD_SET(fd1, &readfs); /* set testing for
        source 1 */
        FD_SET(fd2, &readfs); /* set testing for
        source 2 */
        /* block until input becomes available */
        select(maxfd, &readfs, NULL, NULL, NULL);
        if (FD_ISSET(fd1)) /* input from
        source 1 available */
            handle_input_from_source1();
        if (FD_ISSET(fd2)) /* input from
        source 2 available */
            handle_input_from_source2();
    }
}
```